WELT 4 · DOPPELSTUNDEN 26 UND 27 · IDE-LEVEL 4.3 UND 4.4

W4·2 – Unverändert lassen und die Ladeliste

Datei: `AQ_W4_L4_Hafen_Geruest.txt` (Klasse Unveraendert: Level 4.3 · Klasse Ladeliste: Level 4.4) · Im Lab: **Level 4.3** und **Level 4.4** · Teil 1 in DS 26, Teil 2 in DS 27

Inventur im Hafen! Die Hafenmeisterin will wissen, wie schwer die Kistentürme sind, ob eine bestimmte Kiste im Lager liegt und welche Kiste am Kai die schwerste ist. Die Regel des Hafens ist streng: **Nach der Inventur muss jeder Turm und jede Schlange wieder genau so aussehen wie vorher.** Außerdem bekommt der Hafen eine **Ladeliste**, in die Kisten an jeder Stelle eingefügt und aus der sie an jeder Stelle entfernt werden können.

Beides ist Abitur-Standard: „Nach Ausführung der Operation muss der Stapel unverändert sein“ steht in den Aufgaben von 2023 und 2025, sortiertes Einfügen und das Entfernen des Minimums in einer dynamischen Reihung kamen 2024, 2025 und 2026 vor.

Teil 1 – Das Muster „unverändert lassen“ (DS 26)

Infoteil 1: Warum ist „unverändert“ so schwierig?

- Stapel und Schlange werden als **Referenz** übergeben: Die Methode arbeitet auf **demselben** Stapel wie der Aufrufer. Jedes `pop()` verändert also auch „seinen“ Stapel.
- Auf Stapel und Schlange gibt es keinen Zugriff auf die Mitte. Wer alle Elemente auswerten will, **muss sie entnehmen** – und hinterher wieder zurücklegen.
- **Unverändert** heißt: **dieselben Objekte in derselben Reihenfolge**. Neue Kisten mit gleichem Namen und Gewicht zählen nicht – die Tests prüfen mit `==`, ob es dieselben Kisten sind.
- **Rezept:** auswerten – zwischenlagern – zurückschichten – **erst dann** das Ergebnis zurückgeben. Ein `return` mitten in der Schleife lässt die Struktur halb abgebaut zurück.

Infoteil 2: Drei Muster (Struktogramme nach EH § 5)

gesamtGewicht(s: Stapel vom Inhaltstyp Kiste): Ganzzahl
hilf ← erzeuge Stapel
summe ← 0
solange nicht s.isEmpty()
k ← s.pop()
summe ← summe + k.getGewicht() *// auswerten*
hilf.push(k) *// DIESELBE Kiste zwischenlagern*
solange nicht hilf.isEmpty()
s.push(hilf.pop()) *// zurückschichten*
gib summe zurück *// erst jetzt!*

Muster 1: Stapel über einen Hilfsstapel – zweimal umgedreht ist wieder richtig herum

anzahlLeichter(q: Schlange vom Inhaltstyp Kiste, grenze: Ganzzahl): Ganzzahl	
hilf ← erzeuge Schlange	
anzahl ← 0	
solange nicht q.isEmpty()	
k ← q.dequeue()	
k.getGewicht() < grenze	
wahr	falsch
anzahl ← anzahl + 1	–
hilf.enqueue(k)	
solange nicht hilf.isEmpty()	
q.enqueue(hilf.dequeue())	
gib anzahl zurück	

Muster 2: Schlange über eine Hilfsschlange – die Reihenfolge bleibt erhalten

Frachter
- ladung: Schlange vom Inhaltstyp Kiste - anzahl: Ganzzahl // Kisten in ladung
+ c Frachter() + laden(k: Kiste): nichts + anzahlUeber(grenze: Ganzzahl): Ganzzahl …

anzahlUeber(grenze: Ganzzahl): Ganzzahl	
zahl ← 0	
für i von 1 bis anzahl	
k ← ladung.dequeue()	
k.getGewicht() > grenze	
wahr	falsch
zahl ← zahl + 1	–
ladung.enqueue(k) *// hinten wieder anstellen*	
gib zahl zurück	

Muster 3: Rotieren mit Zähler – die Klasse Frachter kennt die Anzahl ihrer Kisten

- **Muster 3:** Jede Kiste wird vorne entnommen und sofort hinten wieder angestellt. Nach **genau anzahl** Runden steht die Schlange wieder wie vorher. Ohne bekannte Anzahl gäbe es kein Ende (Endlosschleife). Ist anzahl = 0, läuft die Schleife gar nicht – der Zähler ersetzt hier die Leer-Abfrage, aber nur, wenn er stimmt.
- In Struktogrammen stehen **keine Casts**. In Java schreibst du `Kiste k = (Kiste) s.pop();`.

Aufgabe 1: Inventur im Lagerhaus

PAPIER ★

Der Stapel s enthält (oben zuerst) Wein 6, Salz 3, Erz 5, Holz 3 (Gewicht in kg). Es wird `gesamtGewicht(s)` nach Muster 1 aufgerufen.

a) Verfolge den Aufruf: eine Zeile pro Schleifendurchlauf. **[4 BE]**

Phase	k	summe	s (oben → unten)	hilf (oben → unten)
Start				

b) Ein Teammitglied lässt die zweite Schleife weg. Gib die Rückgabe und s nach dem Aufruf an. **[1 BE]**

c) Ein anderes schreibt `hilf.push(new Kiste(k.getName(), k.getGewicht()));`. Erläutere, warum s danach nicht unverändert ist, obwohl jede Ausgabe gleich aussieht. **[2 BE]**

Aufgabe 2: Die Umkehr-Maschine

PAPIER ★★

Ein Stapel s enthält A, B, C (oben zuerst), eine Schlange q enthält A, B, C (vorne zuerst). Jedes Verfahren schichtet **alle** Elemente in eine leere Hilfsstruktur um und danach **alle** wieder zurück.

a) Ergänze die Tabelle. **[4 BE]**

Verfahren	Hilfsstruktur nach dem Umschichten	Ergebnis	umgedreht?
(1) Stapel → Hilfsstapel → Stapel			
(2) Stapel → Hilfsschlange → Stapel			
(3) Schlange → Hilfsschlange → Schlange			
(4) Schlange → Hilfsstapel → Schlange			

b) Formuliere eine Regel: Wann kommt die alte Reihenfolge heraus, wann die umgekehrte? **[1 BE]**

c) Ein Lehrling wertet eine Schlange aus und lagert die Elemente dabei in einem Hilfsstapel zwischen. Was muss er zusätzlich tun, damit die Schlange am Ende unverändert ist? **[2 BE]**

Aufgabe 3: Papier-Compiler: Liegt die Kiste im Lager?

PAPIER → IDE ★★

Implementiere **von Hand** `enthaelt(s: Stapel vom Inhaltstyp Kiste, name: Zeichenkette): Wahrheitswert` (Level 4.3, Erweiterung): wahr, wenn eine Kiste mit diesem Namen im Stapel liegt. s muss danach unverändert sein – **auch dann, wenn die Kiste schon gefunden wurde**.

Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (Kasten am Ende des Blatts).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte im Testrunner `L43UnveraendertTest` die Tests `erweiterung_enthaelt` und `erweiterung_enthaelt_obenUndUnten`. Beide grün: geschafft – notiere, ob es beim ersten Versuch geklappt hat.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

a) Schreibe die Methode in Java (mit Leer-Abfrage, Cast und equals). **[5 BE]**

b) Ein Mitschüler schreibt in seiner Schleife `if (k.getName().equals(name)) { return true; }`. Gib für die Standard-Ladung s = Wein, Tuch, Salz, Gold, Erz, Holz (oben zuerst) und name = "Gold" den Inhalt von s nach dem Aufruf an. **[2 BE]**

Fehlerlogbuch

Nr.	Was war falsch?	Art: Syntax / Logik / Abi-Check	So vermeide ich es
1			
2			
3			

Aufgabe 4: Abi-Format: Die schwerste Kiste am Kai

PAPIER ★★★

Die Operation `maxGewicht(q: Schlange vom Inhaltstyp Kiste): Ganzzahl` liefert das größte Gewicht der Kisten in q, für eine leere Schlange den Wert −1. Nach Ausführung der Operation muss q unverändert sein.

a) Entwickeln Sie ein Struktogramm für die Operation maxGewicht. Laufzeitfehler sind durch geeignete Abfragen auszuschließen. **[4 BE]**

b) Stellen Sie die Verarbeitung des Aufrufs für q = Erz 5, Tuch 8, Salz 3, Gold 8 (vorne zuerst) in einer Tabelle dar. **[3 BE]**

Phase	k	max	q (vorne → hinten)	Hilfsstruktur
Start				

c) Angenommen, die Anzahl n der Kisten in q ist bekannt. Beschreiben Sie, wie maxGewicht dann ohne Hilfsschlange auskommt, und begründen Sie, warum q auch so unverändert bleibt. **[2 BE]**

Aufgabe 5: Level 4.3 – Unverändert

IDE ★ – ★★★

Arbeite in der Klasse **Unveraendert** (static), Testklasse `L43UnveraendertTest`. **Alle** Methoden lassen die übergebene Struktur unverändert – außer entferneAlle.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	zaehle(s: Stapel): Ganzzahl summe(q: Schlange vom Inhaltstyp Ganzzahl): Ganzzahl	`basis_`
★★ Erweiterung	enthaelt(s: Stapel vom Inhaltstyp Kiste, name: Zeichenkette): Wahrheitswert durchschnittsGewicht(q: Schlange vom Inhaltstyp Kiste): Kommazahl schwerste(s: Stapel vom Inhaltstyp Kiste): Kiste	`erweiterung_`
★★★ Experte	unterste(s: Stapel vom Inhaltstyp Kiste): Kiste entferneAlle(q: Schlange vom Inhaltstyp Kiste, name: Zeichenkette): Ganzzahl	`experte_`

- Die Testmeldungen zeigen den ganzen Inhalt der Struktur vorher und nachher – vergleiche Kiste für Kiste.
- `durchschnittsGewicht`: Kommazahl-Division und leere Schlange (→ 0.0) beachten. `schwerste`: bei Gleichstand die **oberste**.
- `entferneAlle` verändert q absichtlich: Alle Kisten mit dem Namen verschwinden, die übrigen behalten ihre Reihenfolge.

🎯 Im Lab üben

- **Level 4.3 „Inventur: Struktur unverändert lassen“** – passt zu Teil 1, Aufgaben 1–5
- **Trainingsplatz Welt 4:** Zufallsaufgaben zu Stapel, Schlange und dynamischer Reihung (1 Punkt je Aufgabe)

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit deinem Namen und deinem persönlichen Code.

Teil 2 – Die Ladeliste: dynamische Reihung (DS 27)

Stapel und Schlange sind streng: nur oben, nur vorne. Die Ladeliste des Hafens muss mehr können – Kisten werden nach Gewicht **einsortiert**, aus der **Mitte** entnommen und über ihre **Position** gefunden. Dafür gibt es die **dynamische Reihung**: wie eine Reihung mit Index, aber ihre Länge wächst und schrumpft.

Infoteil 3: Die dynamische Reihung nach EH § 4

dynamische Reihung (DynArray)	Wirkung
c DynArray()	erzeugt eine leere dynamische Reihung
isEmpty(): Wahrheitswert	wahr, wenn die Reihung kein Element enthält
getLength(): Ganzzahl	Anzahl der Elemente
getItem(i: Ganzzahl): Inhaltstyp	liefert das Element an Position i – nur für 0 ≤ i < getLength()
append(x: Inhaltstyp): nichts	hängt x hinten an
insertAt(i: Ganzzahl, x: Inhaltstyp): nichts	fügt x an Position i ein, die folgenden Elemente rücken nach hinten – für 0 ≤ i ≤ getLength() (i = getLength() hängt an)
setItem(i: Ganzzahl, x: Inhaltstyp): nichts	ersetzt das Element an Position i – nur für 0 ≤ i < getLength()
delete(i: Ganzzahl): nichts	entfernt das Element an Position i, die folgenden rücken nach vorn – nur für 0 ≤ i < getLength()

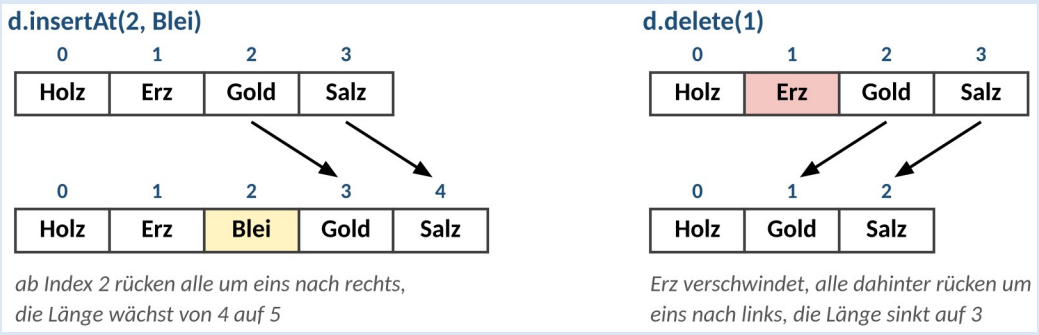


Abbildung 1: insertAt verschiebt nach hinten, delete nach vorn – die Indizes ändern sich!

In Java: `DynArray d = new DynArray();` und beim Lesen casten: `Kiste k = (Kiste) d.getItem(i);`. In Signaturen und Struktogrammen heißt der Typ „dynamische Reihung vom Inhaltstyp Kiste“; dort schreibst du einfach `d.getItem(i).getGewicht()`.

Abi-Check „Index?“

- Vor getItem, setItem und delete: **0 ≤ i < d.getLength()**; insertAt darf zusätzlich i = getLength() (Anhängen).
- Schleifen: `for (int i = 0; i < d.getLength(); i++)` – die Länge **in jeder Runde neu** abfragen, sie kann sich durch delete ändern.
- Wer etwas entfernen **und** zurückgeben will: **erst merken** (getItem), **dann löschen** (delete).
- Leere Reihung: `d.getItem(0)` ist ein Laufzeitfehler – vorher `d.isEmpty()` prüfen.

Aufgabe 6: Die Ladeliste wächst und schrumpft

PAPIER ★

Die Containernummern stehen in einer dynamischen Reihung d vom Inhaltstyp Ganzzahl.

```
1  DynArray d = new DynArray();
2  d.append(18);
3  d.append(42);
4  d.insertAt(1, 7);
5  d.append(42);
6  d.insertAt(0, 3);
7  d.setItem(2, 137);
8  d.delete(1);
9  d.insertAt(d.getLength(), 5);
```

a) Gib den Inhalt von d nach jeder der Zeilen 2 bis 9 an. **[4 BE]**

nach Zeile	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5
2						
3						
4						
5						
6						
7						
8						
9						

b) Gib die Ausgabe von `println(d.getLength() + " " + d.getItem(2));` nach Zeile 9 an. **[1 BE]**

---

c) Implementiere `ersetze(d: dynamische Reihung vom Inhaltstyp Ganzzahl, alt: Ganzzahl, neu: Ganzzahl): Ganzzahl`. Die Methode ersetzt jeden Wert alt durch neu und liefert die Anzahl der Ersetzungen. Gib die Rückgabe von `ersetze(d, 42, 99)` für d nach Zeile 9 an. **[3 BE]**

Aufgabe 7: Sortiert verladen

PAPIER ★★

Die Ladeliste d ist **aufsteigend nach Gewicht** sortiert: d = Salz 3, Erz 5, Wein 6, Tuch 8 (Index 0 zuerst). Die Operation `einfuegenSortiert(d, k)` fügt die Kiste k so ein, dass d sortiert bleibt – **hinter** allen Kisten mit gleichem Gewicht.

a) Gib d an nach `einfuegenSortiert(d, Blei 6)`, danach nach `einfuegenSortiert(d, Kork 1)` und danach nach `einfuegenSortiert(d, Anker 9)`. **[2 BE]**

b) Entwickeln Sie ein Struktogramm für die Operation `einfuegenSortiert(d: dynamische Reihung vom Inhaltstyp Kiste, k: Kiste): nichts`. Laufzeitfehler sind auszuschließen. **[4 BE]**

c) Dokumentieren Sie den Ablauf Ihres Verfahrens für den Aufruf mit Blei 6 (Ausgangsliste wie oben). **[2 BE]**

pos	d.getItem(pos)	Aktion

d) Begründen Sie, dass Ihr Verfahren auch für eine leere Liste und für eine Kiste, die schwerer als alle anderen ist, ohne Laufzeitfehler arbeitet. **[1 BE]**

Aufgabe 8: Die schwerste Kiste zuerst ausladen

PAPIER ★★

Die Operation `entferneSchwerste(d: dynamische Reihung vom Inhaltstyp Kiste): Kiste` entfernt die schwerste Kiste aus d (bei Gleichstand die **erste** davon) und liefert sie. Für eine leere Liste liefert sie null.

a) Implementiere die Operation in Java. **[4 BE]**

b) d enthält die Standard-Ladung Holz 3, Erz 5, Gold 8, Salz 3, Tuch 8, Wein 6. Gib Rückgabe und d nach dem ersten, zweiten und dritten Aufruf an. **[2 BE]**

c) Jemand beendet die Methode mit `d.delete(maxIndex); return (Kiste) d.getItem(maxIndex);`. Welche Kiste liefert dann der erste Aufruf aus b)? Erläutere den Fehler und nenne eine Liste, bei der diese Fassung sogar einen Laufzeitfehler auslöst. **[2 BE]**

Aufgabe 9: Die Kiste, die übersprungen wurde

PAPIER ★★

Ziel: alle B löschen – Vorwärtsschleife mit i++ nach delete

	0	1	2	3	
i = 1:	A	B	B	C	B gefunden → delete(1)
	0	**1**	**2**		
danach:	A	B	C		das zweite B ist auf Index 1 nachgerückt …
	0	**1**	**2**		
i = 2:	A	B	C		**… aber i ist schon 2: C wird geprüft – B wurde ÜBERSPRUNGEN**

Abbildung 2: Löschen in einer Vorwärtsschleife – der Nachfolger rückt nach und wird übersprungen

```
// soll alle Kisten mit diesem Namen aus d entfernen
public static void entferneName(DynArray d, String name) {
    for (int i = 0; i < d.getLength(); i++) {
        Kiste k = (Kiste) d.getItem(i);
        if (k.getName().equals(name)) {
            d.delete(i);
        }   // → Tabellenzeile
    }
}
```

a) d = Salz, Holz, Holz, Erz, Holz (Index 0 zuerst). Verfolge den Aufruf `entferneName(d, "Holz")`: eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle. **[3 BE]**

i	d.getLength()	Name an Index i	Aktion	d danach

b) Erläutere den Fehler allgemein. Bei welchen Listen tritt er auf, bei welchen nicht? **[2 BE]**

c) Korrigiere die Methode auf zwei Arten: (1) mit einer **rückwärts** laufenden Zählschleife, (2) mit einer while-Schleife, die i **nur dann erhöht, wenn nicht gelöscht** wurde. **[3 BE]**

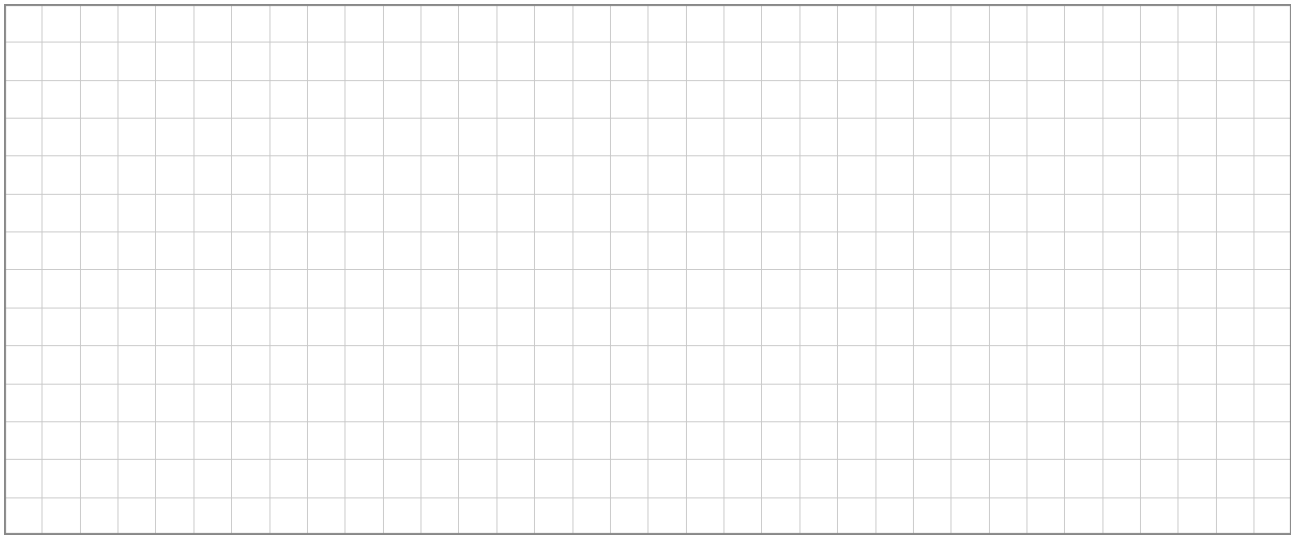
Aufgabe 10: Doppelt geladen

PAPIER ★★★

Die Operation `entferneDuplikate(d: dynamische Reihung vom Inhaltstyp Kiste): nichts` entfernt Kisten mit gleichem Namen: Nur das **erste** Vorkommen bleibt, die Reihenfolge der übrigen bleibt erhalten.

a) Gib d nach dem Aufruf für d = Holz 3, Erz 5, Holz 4, Holz 2, Salz 3, Erz 6 an. **[1 BE]**

b) Entwickle ein Struktogramm für entferneDuplikate. Tipp: Außen läuft i über alle Kisten, innen j ab i + 1 über die Kisten dahinter. **[4 BE]**



c) Wie viele Namensvergleiche braucht dein Verfahren für die Standard-Ladung (sechs verschiedene Namen)? Wie viele sind es allgemein bei n verschiedenen Namen? **[2 BE]**

Aufgabe 11: Level 4.4 – Die Ladeliste

IDE ★ – ★★★

Arbeite in der Klasse **Ladeliste** (static), Testklasse `L44LadelisteTest`. d steht immer für eine dynamische Reihung vom Inhaltstyp Kiste.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	gesamtGewicht(d): Ganzzahl indexVon(d, name: Zeichenkette): Ganzzahl leichteste(d): Kiste	`basis_`
★★ Erweiterung	einfuegenSortiert(d, k: Kiste): nichts entferneLeichteste(d): Kiste entferneDuplikate(d): nichts	`erweiterung_`
★★★ Experte	einfuegesortierung(d): nichts (stabil, nur getItem/setItem/insertAt/delete) abspalten(d, grenze: Ganzzahl): dynamische Reihung vom Inhaltstyp Kiste	`experte_`

- **Papier first:** Deine Lösungen aus den Aufgaben 7 b), 8 a) (gespiegelt: leichteste statt schwerste) und 10 b) sind fast fertige Methoden – abtippen und testen.
- Meldet ein Test „LAUFZEITFEHLER: Index außerhalb“, prüfe die Schleifengrenze – und ob du nach einem delete den Index trotzdem erhöht hast.

Für Schnelle: Sortiert? Sortieren mit Stapeln!

- **istAufsteigend(d: dynamische Reihung vom Inhaltstyp Kiste): Wahrheitswert** – wahr, wenn keine Kiste leichter ist als ihre Vorgängerin (leere Liste: wahr).
- **Stapel sortieren:** Sortiere einen Stapel vom Inhaltstyp Ganzzahl, z. B. s = 3, 8, 1, 5 (oben zuerst), sodass am Ende der kleinste Wert oben liegt. Erlaubt sind nur Stapel und einzelne Variablen – keine Reihung, keine dynamische Reihung.
- **Rotieren um k:** Schreibe `rotiere(q: Schlange, k: Ganzzahl): nichts`, die die vordersten k Elemente nacheinander hinten anstellt. Was passiert für k größer als die Länge von q?

🎯 Im Lab üben

- **Level 4.4 „Ladeliste: dynamische Reihung“** – passt zu Teil 2, Aufgaben 6–10

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit deinem Namen und deinem persönlichen Code.

Abi-Check „unverändert?“ und „Index?“

- ☐ Struktur unverändert, falls gefordert: Stapel über Hilfsstapel, Schlange über Hilfsschlange oder Rotieren mit bekannter Anzahl – **dieselben** Objekte zurücklegen.
- ☐ Ergebnis in einer Variablen merken, `return` erst **nach** dem Zurückschichten.
- ☐ Vor pop/top/dequeue/head **Leer?**; vor getItem/setItem/delete **0 ≤ i < getLength()**, insertAt bis getLength().
- ☐ Löschen in einer Schleife: rückwärts laufen oder i nur erhöhen, wenn nicht gelöscht wurde.
- ☐ Erst merken (getItem), dann löschen (delete); bei Gleichstand auf < bzw. > statt ≤ bzw. ≥ achten.